

The Harness and the Brief

Become an AI-Native PM · Session 2 · Student handbook

WHAT'S IN HERE

- | | |
|----------------------------------|------------------------------------|
| 1. How to use this | 7. 5 · Install the packs |
| 2. 0 · Before we start | 8. 6 · Write a skill |
| 3. 1 · Kick off the research | 9. 7 · The research, and the brief |
| 4. 2 · GitHub and the repository | 10. 8 · Homework |
| 5. 3 · Standing instructions | 11. 9 · Going further |
| 6. 4 · Memory | 12. 10 · If you fall behind |

How to use this

Every command and every prompt from tonight is in here, in the order we run them. Follow along live if you can keep up, and use this to catch up if you cannot. Nothing in this session requires you to improvise or to type anything from memory.

Each section has three parts: what you are doing, the exact thing to run, and what to do when it does not work. Read the third part only when you need it.

Tonight produces no product code. You leave with a repository, a working harness, and a brief. The code starts in Session 4.

0 Before we start

Open a terminal and run:

```
claude
```

If it starts and you are logged in, you are ready. Quit back out with `/exit`.

If it does not start, post in the thread immediately rather than waiting. We will sort it out while the session continues, and you will not fall behind.

1 Kick off the research

What you are doing. Redline is a shape, not a validated product. Before you write a brief, you want evidence about who actually gets hurt by contracts and what already exists. Four agents go and find out while the rest of the session runs.

Make a folder and start Claude in it:

```
mkdir redline && cd redline
claude
```

Then paste this whole prompt:

```
I'm building a product called Redline. It reads a contract, lease, freelance
agreement or terms of service and tells the reader what they're actually
signing: a plain-English summary, risky clauses ranked by severity with the
exact source sentence shown, a drafted counter-offer for each one, and a
question box that answers only from the document.
```

```
That is a hypothesis, not a validated product. Before I write a PRD I want it
anchored in real pain.
```

```
Run four research agents IN PARALLEL. Use the Sonnet model for every one of
them. Do not do the research yourself in this session.
```

```
Agent 1, who has this pain: find real people describing being hurt by contract
terms they did not understand or did not notice. Public forums and communities.
Capture verbatim quotes and where each one came from.
```

```
Agent 2, what goes wrong: identify the recurring clause types that actually
```

burn people. Auto-renewal, unilateral termination, IP assignment, liability caps, indemnity, non-competes, fee escalators, and whatever else the evidence shows. Rank them by how often they show up in complaints.

Agent 3, what already exists: find existing contract-review and clause-analysis products. For each one, what it does, who it targets, what it charges, and the most common complaint about it. Include both free and AI-native ones.

Agent 4, who would pay: identify the segments feeling this most sharply, and any evidence about willingness to pay. Stated prices, what people currently spend on a lawyer for this, what they call too expensive.

Guardrails, for every agent:

- Hard cap of 12 web searches and 15 pages read. Stop at whichever comes first.
- Stop early once you have 8 distinct sourced findings. More is not better.
- Every finding carries a source URL. If you cannot source it, drop it.
- Do not speculate to fill gaps. Say what you could not find.
- Write only to research/<agent-name>.md. No code, no other files.

When all four are done, write research/summary.md with:

- The three sharpest pain points, each with a real quote and its source
- The clause types that matter most, ranked
- Where the existing tools are weak
- Who would plausibly pay, and roughly what
- Anything that contradicts my hypothesis. If the evidence does not support building this, say so plainly.

Then stop and tell me you're done. Do not start designing or building anything.

Now leave this terminal alone for the rest of the session. Everything else tonight happens in a second window.

If it goes wrong. If the parallel run fails or stalls, run the same prompt with "Run four research agents IN PARALLEL" changed to "Research these four questions one at a time." It is slower and it still works. If nothing comes back at all, a checkpoint summary is posted in the thread and you can use that instead. You need this at section 7, not before.

2 GitHub and the repository

What you are doing. Putting the folder under version control and pushing it to GitHub. This is one of the three things you walk out of the course with, and it is also what makes it safe to let an agent make big changes later.

Open a second terminal window and move into the same folder. Your research is still running in the first one.

Install the GitHub CLI

This is separate from git. Git tracks versions on your machine. The CLI is what lets your machine, and Claude, talk to GitHub for you.

YOUR MACHINE	RUN THIS
macOS	<code>brew install gh</code>
Windows	<code>winget install --id GitHub.cli --source winget</code>
Linux	See cli.github.com/manual/installation

On Windows, close your terminal and open a new one afterwards. The installer changes your PATH and the change does not apply to a window that was already open. Skip this and `gh` will look like it failed to install when it did not.

Log in

Run this yourself, in your terminal. **Do not ask Claude to run it.** It asks interactive questions that Claude cannot answer, and it will simply hang.

```
gh auth login
```

Answer, in order:

- 1. GitHub.com
- 2. HTTPS
- 3. Authenticate with a web browser

4. Copy the one-time code it shows you, press Enter, and paste it in the browser

Tell git who you are

This is a different thing from logging in. It sets the name and email stamped onto your commits. It grants no access to anything. Confusing these two is the most common mistake in git setup.

```
cd redline
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git init
```

Hand the rest to Claude

Start Claude in this second terminal, then paste:

This folder will hold a product called Redline. Set it up as a git repository and push it to my GitHub.

1. Write a .gitignore suitable for a Next.js project. It must exclude .env.local, .env, node_modules, and anything else that could leak a credential later.
2. Show me the list of files that will be included in the first commit so I can check nothing sensitive is in it. Then commit everything currently in the folder. A research folder is being written by another session right now, so include whatever exists and we will commit the rest later.
3. Create the GitHub repository and push, using exactly:
gh repo create redline --public --source=. --push
If gh is not authenticated, stop, tell me to run `gh auth login` myself, and wait for me.
4. Print the repository URL when you're done.

Do not write any application code. There is nothing to build yet.

If it goes wrong.

- `brew: command not found` on macOS means you do not have Homebrew. Either

install it from brew.sh, or download the installer directly from github.com/cli/cli/releases.

- `gh: command not found` on Windows after installing almost always means you

did not open a new terminal. Do that first before anything else.

- If the push is rejected, tell Claude exactly what the error said. It will

fix it. Do not try to fix it by hand.

3 Standing instructions

What you are doing. Writing CLAUDE.md, the file Claude reads at the start of every session in this project, without being asked.

The rule for what goes in: only things it could not work out by reading the files. For every line, ask whether leaving it out would cause a mistake. If it would not, leave it out.

Keep it under 60 lines. Anthropic's own guidance is under 200, and the teams who do this best run under 60. An overlong file does not just cost tokens, it measurably reduces how reliably any of the rules get followed.

One thing worth knowing, because most advice online gets it backwards: pointing at another file with `@` does **not** save you context. Imports load at launch exactly like the text they replace. A plain sentence naming a file is different, and it is the move worth learning. "The research lives in research/summary.md, read it before deciding what the product should do" costs one line and gets opened only when it matters.

Write a CLAUDE.md for this project. Hard limit of 60 lines. Include only what you could not work out by reading the files in this repo, and make it hold up during a long build that runs while I am not watching.

Apply this test to every line before you write it: would leaving it out cause you to make a mistake? If it would not, leave it out.

The project: Redline. A web app where someone uploads a contract, lease, freelance agreement or terms of service and gets back a plain-English summary, the clauses that could hurt them ranked by severity with the exact source

sentence shown, a drafted counter-offer for each flagged clause, a question box that answers only from the document, an editable list of their own red lines that drives the analysis, and a saved library of past documents.

Settled decisions, not open for reinterpretation:

- Next.js, Supabase for auth and database, deployed on Vercel.
- The uploaded file is parsed in the browser. Only text is stored.
- Every risk flag cites the exact sentence it came from. A flag whose source cannot be shown is a bug.
- The product calls its model through OpenRouter.

Scope: build the capabilities listed above and stop there. When something looks like the obvious next step and is not on that list, ask me first. Payments, billing, OCR for scanned documents, and sharing a document between users are excluded on purpose. This version exists to prove the analysis can be trusted, and none of those three make it more trustworthy. OCR would actively undermine it, because a citation is worthless when the text it points at was misread.

Standing rules, written as what to do:

- Keep credentials in .env.local, which is gitignored. Never commit a secret, because a key is public the moment it is pushed and has to be rotated.
- State only what the document says. Where the text does not support a claim, the product does not make it.
- Ask me before adding a dependency.

Write these as one-line pointers rather than pasting their contents, so the file stays short and you open them only when they matter:

- research/summary.md holds the user research. Read it before deciding what the product should do.
- The brief will live in PRD.md once it exists. Read it before building.

Use plain markdown headers. Ask me anything you need first, then write the file and show it to me.

When it shows you the draft, cut it. Delete two or three lines. The cutting is the part that matters and it is the part everyone skips.

Later in the session, one of the tools you install will add a short `## Agent skills` section to this file. That is expected. It appends and leaves your own writing alone.

4 Memory

What you are doing. Looking at the memory layer you already have, then writing your first decision into the one that actually ships.

Run this and look around:

```
/memory
```

Claude has been taking notes on how you work since your first session. That is auto memory: Claude writes it, you did not. It lives on this laptop, in `~/.claude/projects/<project>/memory/`.

The thing to understand is that **auto memory is not in your repository**. It does not follow you to another machine and nobody who clones your project gets any of it. So anything the project needs to remember goes in the repo. Anything about how you personally like to work can stay where Claude put it.

If your folder is nearly empty, that is expected. You have had Claude Code for a week.

Now write the first decision:

```
Create the first architecture decision record for this project, at  
docs/adr/0001-every-flag-cites-its-source.md.
```

```
The decision: every risk flag Redline produces must cite the exact sentence  
from the uploaded document it came from. A flag whose source sentence cannot  
be shown is treated as a bug rather than a formatting preference.
```

```
Write it in four short sections:
```

- Decision: what we are doing.
- Alternatives: what else we could have done, including letting the model describe risks in its own words without quoting anything.
- Why: what this buys us, expressed as what a reader of the output can go and verify for themselves.
- Consequences: what this now rules out or makes harder, for the product and

```
for how it gets built and tested.
```

Keep it under 40 lines. Ask me first if you need anything. Do not write any application code.

Look at what that one decision rules out. Scanned documents. Paraphrased summaries. A model that is careless about copying text back accurately. Three product decisions out of one, which is why the consequences section is the part that does the work.

5 Install the packs

What you are doing. Installing two things other people built better than you would have, then pointing them at this project.

```
/plugin install mattpocock-skills@claude-plugins-official  
/plugin install skill-creator@claude-plugins-official
```

If the install summary tells you to, run:

```
/reload-plugins
```

Then set them up for this repo:

```
/mattpocock-skills:setup-matt-pocock-skills
```

This one is a conversation, not a script. It will ask you questions. Answer them like this so the whole cohort ends up on one configuration:

IT ASKS	YOU ANSWER
Where do issues live?	Local markdown
Keep the default triage labels?	Yes
Here is what I will write	Accept

Note what we are not installing. The design pack and the writing pack are both good and neither is needed until Session 3, so they go in during Session 3. Every plugin's skills get read at the start of every session, so you pay for them constantly and use them rarely. Install a thing when you need it, not when you first hear about it.

Commit

Write the message yourself rather than letting it guess. The diff already shows what changed. The message is the only place you get to say why.

```
Commit everything we've added since the first commit. Use exactly this
message:
```

```
Add project harness: instructions, first decision, agent config
```

```
CLAUDE.md carries the settled decisions and the scope boundary, so a build
running unattended stays inside it.
```

```
ADR 0001 records why every risk flag must cite its source sentence. That
single decision also rules out OCR and paraphrased summaries, which is why
it was worth writing down before any code exists.
```

```
docs/agents/ tells the engineering skill pack where issues and decisions
live in this repo.
```

The shape of a good commit message: a short summary line in the imperative with no full stop, then a blank line, then the why. "Add project harness," not "Added" and not "Adding." The test is whether the message still makes sense to you in six weeks, in an interview, having forgotten the session entirely.

If it goes wrong. If `/plugin install` cannot find the marketplace, run `claude plugin marketplace add anthropics/claude-plugins-official` first, then try again.

6 Write a skill

What you are doing. Building `/flush`, a session-capture skill. This one is yours rather than this project's. You will use it on everything you build after this course.

Something earns being a skill when you have done it twice and expect to do it again. Before you write one, check you do not already have it. Two skills competing for the same request makes both of them fire less reliably.

Step one, design it

```
/mattpocock-skills:grill-me
```

I want to build a skill called flush. At the end of a working session it reads the conversation and files whatever is worth keeping into the right place in the repo, so nothing durable is lost when the session ends.

Grill me on what it should and should not do. One round only, at most five questions. Focus on: what counts as worth keeping, where each kind of thing goes, what it should refuse to write, and how it should behave in a repo that has none of the folders it expects.

Step two, build it

```
/skill-creator:skill-creator
```

Build the skill we just designed. Create it at `~/.claude/skills/flush/` so it is available in every project rather than only this one.

It runs at the end of a working session, reads the conversation, and files whatever is durable into the repo.

Use what I said in the interview above, plus these:

- Before writing anything, look at what this repo actually has. Check for a decisions folder, a glossary file, an issue tracker, and a session log

directory. Route to whatever exists. Where something is missing, say what you would create and ask before creating it.

- Propose changes to CLAUDE.md rather than making them. Show the change and wait for a yes.
- Never write a credential, an API key, or the contents of a .env file.
- When nothing durable happened, say so and write nothing.

Write the description so it fires when I say flush, capture this session, or save what we decided, and not otherwise.

The "look before it writes" instruction is what makes this portable. Your next project will not have these folders. A skill that assumes a layout works once. A skill that checks what exists works everywhere.

Step three, check when it fires

Run an eval on the flush skill's description. Tell me where it triggers when it should not, and where it fails to trigger when it should.

A skill that does the right thing at the wrong moment is worse than no skill, because you stop trusting it. The description is not documentation. It is the trigger.

7 The research, and the brief

What you are doing. Finally reading what came back, then getting interviewed until you have a brief worth building from.

Read it

Open `research/summary.md`.

Read the last section first, the part saying what contradicts the idea. Read that before anything that agrees with you, because you will believe the agreeable parts whether or not they are true.

What you are looking for is narrow. Is the pain real. Is it sharp enough that somebody would go and find a tool for it. Is there a specific version of it that existing products handle badly.

Where the evidence is thin, say so in the brief. A brief that admits what it does not know is worth more than one that papers over it.

The interview

This is the part that matters most tonight. A brief written straight from the skeleton would be identical to everyone else's. Two things make yours different: what you know about risk that nobody else does, and the calls you are willing to make.

A brief a model wrote is comprehensive, balanced, and decides nothing. It lists every option, picks none, and leaves no losers. A brief with judgement in it makes calls someone could disagree with.

```
/mattpocock-skills:grill-with-docs
```

```
Read research/summary.md first, then CLAUDE.md and docs/adr/0001.
```

```
I am writing the brief for Redline. Your job is not to collect requirements from me. It is to make me decide things.
```

```
The difference between a brief a product manager wrote and one a model wrote is that the first contains calls somebody could disagree with. So do not ask me what the product should do. Ask me to choose between things I cannot have both of, push back on the answer, and make me say what I am giving up.
```

```
Hard limit: ten questions across three rounds. Give me your recommended answer alongside each question so I can accept it in one word. After round three, stop and tell me you are done, even if you would normally continue.
```

```
Round one, three questions on who this is for. Force a choice between the segments the research surfaced. I cannot serve all of them in a first version and the brief should name the one I am not serving.
```

```
Round two, three questions on what I know that the research does not. Which clauses I would flag, how severely, and what makes one dangerous rather than merely unusual. If I have no direct experience here, work from the clause types the research surfaced.
```

```
Round three, four questions on trade-offs and blind spots. Cover at least these, in your own words:
```

- Which error I would rather make: flagging something harmless, or missing something that hurts. Both cost something and I have to pick.
- How confident the output should sound when the model is not certain, given that hedged language is safe and useless while confident language is useful and sometimes wrong.
- What should happen when a document is genuinely fine, since most are, and a tool that always finds problems stops being believed.
- One thing I have not considered at all. Tell me what it is rather than asking me to guess it.

Write new vocabulary into CONTEXT.md and any decisions into docs/adr/ as we go.

Write the brief

Write the brief to PRD.md from what we just settled.

Sections:

- Who this is for, specifically, and what they do today instead.
- The problem, with at least one real quote from the research and its source.
- What the first version does, as a numbered list, and nothing beyond it.
- What good looks like: how someone would know whether the analysis is any good, stated specifically enough to test later.
- My red lines: which clauses I flag, how severely, and why each one matters.
- The calls I made and what I gave up. For each one: the choice, what I chose against, and who is worse off because of it.
- What we are not building, and why.
- What the research could not tell us.

Write as much as the detail genuinely requires. Test every line by asking whether removing it would change what gets built or how someone would judge it, and cut the ones where it would not. Quote the research where it supports a claim and say plainly where nothing does. Do not write any application code.

****The "what good looks like" section is the most important thing you write tonight.**** Session 5 builds your eval suite directly from it. Write it specifically enough that someone else could test against it.

Commit

Commit everything from tonight. Use exactly this message:

Add research, brief, and domain vocabulary

PRD.md is the brief for the first version, written against the research in research/ rather than assumed. Its "what good looks like" section is what the eval suite will be built to measure.

CONTEXT.md holds the vocabulary we agreed, so the same words mean the same things in every future session.

8 Homework

Required, before Friday

Session 3 depends on both of these. Neither takes long.

1. Turn your brief into a spec.

Read PRD.md, then run:

```
/mattpocock-skills:to-spec
```

Nothing else. It does not interview you. It synthesises what is already in the file, so you do not need to explain the project to it first.

2. Build one more skill of your own.

Whatever you catch yourself repeating this week. Use the same three steps from section 6: [grill-me](#) to work out what it should do, [skill-creator](#) to build it, then an eval to check when it fires.

Ideas, if nothing comes to mind: a skill that reviews a document against a checklist you keep, one that turns meeting notes into decisions and actions, one that drafts a status update from your week's

commits, one that stress-tests a plan before you send it.

Before Session 4

Get a small amount of credit on OpenRouter. Your product calls a model on its own, from a server, with nobody sitting in a chat window. That runs on an API key rather than on your Claude subscription. A subscription is for you; an API key is for your product.

Go to openrouter.ai, create an account, and add credit. We choose the model together in Session 3, so do not pick one yet.

Never paste an API key into a chat window, a forum post, or a screenshot. If it happens, delete the key and make a new one. It takes ten seconds.

9 Going further

None of this is needed for Session 3. It is here for anyone who wants to push.

A proper memory layer. Claude Code's auto memory is machine-local and it is Claude writing about you. If you want memory that ships with your projects and that you control, there are a few approaches worth reading about. Tuning what you already have is the cheapest option and costs no install. Adopting a library built for it is the middle path. Building the pattern yourself, where the model maintains a linked set of markdown files over immutable sources, is the most work and the most yours.

Branding. We are not covering it live. A product that looks generated reads as generated. Give Claude your product name, ask for three name and colour directions with reasoning, pick one, and write the result into your repo as a short brand note. Fifteen minutes, and it changes how the finished thing lands.

Automating your capture skill. You cannot make a hook call a skill. Hooks run shell commands, and the session-end hook specifically cannot feed anything back to the model, because the session is already over. What you can do is have a session-end hook run a script that processes the transcript separately. It is real automation and it costs tokens silently every time, so it is a deliberate choice rather than a free win.

That distinction is worth carrying beyond this course. A hook is for things that must happen every time regardless of what the model decides. A skill is for things that need judgement. Getting that backwards is one of the most common mistakes people make when specifying AI automation.

10 If you fall behind

Nothing tonight is unrecoverable. Everything is committed and pushed, so a dead laptop costs you nothing.

Post in the thread during the week rather than waiting for Friday. Problems posted on Tuesday get solved on Tuesday. Problems saved for Friday cost you half of Session 3.

Become an AI-Native PM · Session 2 · Every command and prompt from the session. Post in the cohort thread if anything here does not work.